Good — if you've completed Step 8, your basic app flow is likely set up and tested, so the next best step is to do a final functional check and cleanup before moving to project-completion materials. In Thymeleaf, using the iteration status variable like stat.count is the correct way to show serial numbers independent of database primary keys, so your S.No column can safely remain 1, 2, 3 on the page even if stored IDs are 1, 2, 4 after deletion.baeldung+1

What "Done Step 8" means
At this stage, you should have already tested:
Customer CRUD
Product CRUD
Order creation
Stock deduction
Out-of-stock alert
Delete and update flow
UI rendering end to end.github+1
If all of those are working, then your project has reached a working MVP submission state. A standard Spring Boot + Thymeleaf CRUD app is considered working when the app runs, pages render, and add/edit/delete/list flows operate correctly from browser to database and back.baeldung+1
What you should do now
Final verification checklist
Run this exact final check once:
Add 2 customers and confirm they appear in the customer list.baeldung
Add 3 products with different stock values, including one with stock 0, and confirm the out-of-stock product is marked correctly.baeldung
Create an order using an in-stock product and verify stock reduces after saving.baeldung
Try ordering more than available stock and confirm the validation message appears, since stock validation should block invalid orders in service logic.baeldung
Delete one record and confirm the UI serial number still displays sequentially using Thymeleaf loop status rather than database ID.attacomsian+1
Your serial number requirement

For your exact requirement — “if DB IDs become 1, 2, 4 after deletion, frontend should still show proper serial order” — the correct Thymeleaf pattern is:
xml

stat.count gives a 1-based running number for the rendered list, while stat.index starts from 0, so stat.count is the better choice for S.No display.baeldung+1
Recommended next phase
Since you said only “Done Step 8,” the clean next move is not to restart the whole project, but to continue with completion deliverables from your original rules. Based on your own requested workflow, the next items should be generated from the final codebase as:
step-by-step summary
project report
viva questions and answers
resume/project description
GitHub README content
Tell me which one you want next
Reply with any one of these and I’ll continue without restarting the project:
Generate step-by-step summary
Generate project report
Generate viva questions and answers
Generate resume project description
Generate GitHub README
Upgrade this project to support multiple products in one order
If you want, I can now directly start with all 5 completion materials in one go based on this exact project version.

all 5 completion materials in one go based on this exact project version.

Below are all 5 completion materials in one go for your **Order Management System** built with **Spring Boot, Thymeleaf, MySQL, and Spring Data JPA**. This kind of project fits a standard server-side MVC CRUD application structure where Thymeleaf renders the view layer and Spring Data JPA handles persistence and relationships.[1] [2] [3]

Step-by-step summary

This project is a CRUD-based web application developed to manage **customers, products, and orders** with proper relational mapping between tables. The system follows a layered architecture of **Controller → Service → Repository → Database → Thymeleaf UI**, which is a common structure in Spring Boot MVC applications.[2] [4]

Development flow

1. **Project setup** — created a Spring Boot project with dependencies for Web, Thymeleaf, Spring Data JPA, MySQL Driver, and Validation.[1] [2]
2. **Database configuration** — connected the application to MySQL using `application.properties` and enabled Hibernate schema update for development.[3] [1]

3. **Entity design** — created `Customer`, `Product`, `CustomerOrder`, and `OrderItem` entities with one-to-many and many-to-one relationships, which is the correct relational approach for master-detail order data.[5]
4. **Repository layer** — created JPA repository interfaces for each entity to perform database operations without writing manual SQL for normal CRUD flows.[2]
5. **Service layer** — added business logic for customer management, product management, order processing, stock deduction, and out-of-stock validation.[1]
6. **Controller layer** — created controllers to handle routes, load model data, process forms, and return Thymeleaf templates.[3]
7. **Frontend pages** — built Thymeleaf templates for dashboard, customer list/form, product list/form, and order list/form/view.[4] [2]
8. **Testing** — tested add, update, delete, list operations, order placement, stock reduction, and out-of-stock alerts.[1]

Key implemented features

- Customer CRUD
- Product CRUD
- Order creation
- Relational database mapping
- Stock quantity management
- Out-of-stock alert
- Server-side rendering using Thymeleaf
- Frontend serial number using `stat.count` instead of database ID, which is the correct Thymeleaf iteration approach for UI-only numbering.[1]

Project report

Title

Order Management System using Spring Boot, Thymeleaf, MySQL, and Spring Data JPA

Abstract

The Order Management System is a web-based CRUD application developed to manage customer details, product inventory, and customer orders in an integrated manner. The application uses Spring Boot for backend development, Thymeleaf for server-side page rendering, Spring Data JPA for ORM-based database access, and MySQL for persistent storage. The main objective of the project is to demonstrate CRUD operations, entity relationships, form handling, and end-to-end flow in a relational web application.[2] [1]

Introduction

In many business environments, order handling requires structured management of customers, products, and purchase records. Manual handling of such records can lead to inconsistency, poor tracking, and stock-related issues. This project solves that problem by providing a centralized system where users can manage customer and product records, create orders, and automatically update stock availability.[4] [3]

Objectives

- To implement CRUD operations for main entities.
- To demonstrate one-to-many and many-to-one mapping in Spring Data JPA.[5]
- To build a server-side rendered web application using Thymeleaf.[3]
- To implement stock deduction and out-of-stock product alerts.
- To create a neat UI suitable for academic submission and demo.

Technologies used

Technology	Purpose
Spring Boot	Backend application framework [1]
Thymeleaf	Server-side rendering of HTML pages [3]
Spring Data JPA	ORM and repository support [2]
MySQL	Relational database [1]
Bootstrap/CSS	Frontend styling [6]
Maven	Dependency and build management [2]

Modules

Customer module

This module is used to add, update, delete, and view customer details. Each customer can be linked to multiple orders through a one-to-many relationship.[5]

Product module

This module manages product name, category, price, and stock quantity. It also supports stock status indication and out-of-stock alerts.[1]

Order module

This module creates orders by linking a selected customer with a selected product and quantity. The total amount is calculated and stock is reduced after successful order creation. [1]

Order item module

This module stores individual ordered product details such as quantity, unit price, and subtotal. It acts as a child table under the order table and links products to orders properly in a relational format.[5]

System architecture

The system follows the MVC pattern:

- **Model** → JPA Entities
- **View** → Thymeleaf templates
- **Controller** → Spring MVC controllers

The request flow is:
Thymeleaf UI → Controller → Service → Repository → Database → UI, which is consistent with common Spring MVC CRUD applications.[4] [2]

Database relationships

- One Customer → Many Orders
- One Order → Many Order Items
- One Product → Many Order Items

This design supports multiple related tables and foreign key mapping, which is a standard relational approach in order-processing systems.[5]

Features implemented

- Add, edit, delete, and view customers
- Add, edit, delete, and view products
- Create and view orders
- Stock deduction after order placement
- Prevent order when stock is insufficient
- Show out-of-stock products on dashboard and product page
- Display serial number using Thymeleaf loop count instead of database ID

Output

The application provides a dashboard and separate management pages for customers, products, and orders. It enables smooth navigation and clear display of related data using Thymeleaf templates and Bootstrap-based UI.[2] [4]

Conclusion

The project successfully demonstrates a relational CRUD web application using Spring Boot, Thymeleaf, and MySQL. It meets the main academic goals of implementing database relationships, clean layered architecture, server-side rendering, and complete end-to-end functionality.[2] [1]

Viva questions and answers

1. What is Spring Boot?

Spring Boot is a Java framework that simplifies application development by providing auto-configuration, embedded server support, and starter dependencies for common application types.[7]

2. What is Thymeleaf?

Thymeleaf is a Java-based server-side template engine used to render dynamic HTML pages from backend model data.[7] [3]

3. Why did you use Thymeleaf in this project?

Thymeleaf was used because this project is a server-side rendered MVC application, and it integrates naturally with Spring Boot controllers and model attributes.[3]

4. What is Spring Data JPA?

Spring Data JPA provides repository abstraction for database operations and reduces boilerplate code for CRUD and query methods.[2]

5. What is the role of Hibernate in this project?

Hibernate works as the JPA implementation and maps Java entities to database tables.[2]

6. What relationships are used in this project?

This project uses one-to-many and many-to-one relationships between customer and orders, and between order and order items.[5]

7. Why is `OrderItem` needed?

`OrderItem` is needed to store individual product details inside an order, such as quantity, unit price, and subtotal, instead of putting all product data directly into the order table.[5]

8. What is the difference between `stat.index` and `stat.count` in Thymeleaf?

`stat.index` starts from 0, while `stat.count` starts from 1, so `stat.count` is better for serial number display in tables.[1]

9. Why does the frontend serial number not match database ID?

The serial number is only for UI display and is generated dynamically using Thymeleaf loop status, while database ID is the actual primary key and may contain gaps after deletion.[1]

10. How is stock managed in this project?

When an order is created, the ordered quantity is subtracted from product stock in the service layer, and the order is blocked if stock is insufficient.[1]

11. What is MVC?

MVC stands for Model-View-Controller, where Model handles data, View displays UI, and Controller processes requests and responses.[4] [3]

12. Why did you use a service layer?

The service layer is used to keep business logic separate from controllers, making the code cleaner and easier to maintain.[2]

13. What is a repository interface?

A repository interface is a Spring Data JPA interface that provides built-in methods for database operations such as save, find, update, and delete.[2]

14. What happens if a product is out of stock?

The application displays an out-of-stock status and prevents placing an order beyond available stock.[1]

15. What are the main advantages of this project?

It demonstrates CRUD operations, database relationships, layered architecture, form handling, stock management, and server-side rendering in one complete project.[2] [1]

Resume project description

Short resume version

Developed an **Order Management System** using **Spring Boot, Thymeleaf, MySQL, and Spring Data JPA** with full CRUD operations, relational mapping, stock management, out-of-stock alerts, and server-side rendered UI.[2] [1]

Medium resume version

Built a full-stack **Order Management System** using **Spring Boot, Thymeleaf, MySQL, and Spring Data JPA** to manage customers, products, and orders. Implemented CRUD operations, one-to-many and many-to-one relationships, stock deduction logic, out-of-stock alerts, and MVC-based layered architecture with server-side rendered pages.[5] [2]

Strong portfolio version

Designed and developed a relational **Order Management web application** using **Spring Boot, Thymeleaf, MySQL, and Spring Data JPA.** Implemented complete CRUD functionality for customers and products, order processing with stock validation, mapped entity relationships, layered Controller-Service-Repository architecture, and a clean Bootstrap-based UI suitable for academic demo and portfolio showcase.[4] [1]

GitHub README content

Project title

```
# Order Management System
```

Description

```
A CRUD-based web application built using Spring Boot, Thymeleaf, MySQL, and Spring
```

Features

```
## Features

- Customer CRUD operations
- Product CRUD operations
- Order creation and viewing
- One-to-Many and Many-to-One database relationships
- Stock deduction after order creation
- Out-of-stock alerts
- Server-side rendering using Thymeleaf
```

```
- Clean Bootstrap-based UI
- Serial number display using Thymeleaf loop count
```

Tech stack

```
## Tech Stack

- Java
- Spring Boot
- Thymeleaf
- Spring Data JPA
- Hibernate
- MySQL
- Maven
- Bootstrap
```

Project structure

```
## Project Structure

src/main/java/com/example/ordermanagement
├── controller
├── dto
├── entity
├── repository
└── service

src/main/resources
├── static/css
├── templates/customers
├── templates/products
├── templates/orders
└── application.properties
```

Database relationships

```
## Database Relationships

- One Customer can have many Orders
- One Order can have many Order Items
- One Product can appear in many Order Items
```

How to run

```
## How to Run

1. Clone the repository
2. Create MySQL database:
```

```sql
CREATE DATABASE order_management_db;
```

3. Update database username and password in `application.properties`
4. Run the project:

```
mvn clean install
mvn spring-boot:run
```

5. Open in browser:

```
http://localhost:8080/
```

```
### Main pages
```md
## Main Pages

- Dashboard
- Customer List / Add / Edit
- Product List / Add / Edit
- Order List / Create / View
```
```

Key learning outcomes

```
## Learning Outcomes

- Built a Spring Boot MVC application
- Used Thymeleaf for server-side rendering
- Implemented Spring Data JPA repositories
- Managed relational tables with JPA annotations
- Handled business logic through service layer
- Implemented stock validation and alerts
```

Future improvements

```
## Future Enhancements

- Support multiple products in one order
- Add search and filter functionality
- Add authentication and role-based login
- Add low-stock alerts
- Export reports as PDF/Excel
```

If you want, I can next turn these into a more polished **college submission format** with:

1. **formal report with chapter headings**
2. **more viva questions**

3. ATS-friendly resume bullet points
4. professional GitHub README with badges
5. project abstract, objective, scope, and future enhancement section

⁂

1. https://dev.to/tienbku/spring-boot-thymeleaf-crud-example-ijk
2. https://github.com/bezkoder/spring-boot-thymeleaf-example/blob/master/README.md
3. https://www.geeksforgeeks.org/java/spring-boot-thymeleaf-with-example/
4. https://github.com/HongyuNie/Spring-Boot-Spring-MVC-CRUD-MYSQL-Database-Thymeleaf-Poc-Project/blob/main/README.md
5. https://www.youtube.com/watch?v=ctwRpskAeIU
6. https://www.bezkoder.com/spring-boot-thymeleaf-example/
7. https://www.geeksforgeeks.org/springboot/spring-boot-interview-questions-and-answers/
8. https://mindmajix.com/jpa-interview-questions
9. https://www.linkedin.com/pulse/complete-employee-crud-application-spring-boot-thymeleaf-mahto-zbgef
10. https://github.com/ahsumon85/spring-boot-jpa-thymeleaf-mysql